Technical Documentation

March 22, 2026

Contents

1 Clustering

1.1 Objective

The objective of the *clustering* module is to group drivers (identified by their license plate) according to behavioral indicators observed in detection records (`flujos`). These groups are used both for exploratory analysis and to build gantry/interval aggregated variables that feed downstream models.

1.2 Data and preprocessing

We start from a set of observations $\{r_j\}_{j=1}^{N}$, where each record contains, at minimum:

$$r_j = (t_j, p_j, \ell_j, v_j, \text{plate}_j),$$

with t time, p gantry, ℓ lane, v speed, and plate the license plate.

License plate normalization Plates are normalized as uppercase text and invalid values are removed (`""`, `nan`, `null`, `none`). In the implementation, plates with length outside $[5, 6]$ are filtered out.

Lane cleaning and duplicates Lane is converted to numeric and filtered to $\ell \in \{1, 2, 3\}$. In addition, exact duplicates are removed using the key:

$$(\text{plate}, p, \ell, t).$$

Speed outlier handling For each (p, ℓ) group, quantiles q_α and q_β are computed on speed. Depending on the mode:

- **Winsorization**: $v \leftarrow \min(\max(v, q_\alpha), q_\beta)$.
- **Filtering**: observations with $v \notin [q_\alpha, q_\beta]$ are discarded.
- **No action**: v is not modified.

1.3 Driver-level *feature* engineering

For each driver i (license plate), a vector of variables $x_i \in \mathbb{R}^d$ is built from aggregations over their passes. In the implementation, these variables are stored in `Resultados/cluster_features*.duckdb` (table `cluster_features`).

1.3.1 Counts, mean speed, and lane usage

Let N_i be the number of passes of driver i:

$$N_i = \sum_{j=1}^{N} \mathbf{1}[\text{plate}_j = i].$$

Mean speed is defined as:

$$\overline{v}_i = \frac{1}{N_i} \sum_{j:\,\text{plate}_j = i} v_j.$$

For lane usage, we define proportions:

$$\pi_{i,k} = \frac{1}{N_i} \sum_{j:\,\text{plate}_j = i} \mathbf{1}[\ell_j = k], \quad k \in \{1, 2, 3\}.$$

1.3.2 Relative speed

A reference mean speed is defined per 5-minute interval and gantry:

$$\overline{v}_{p,\tau} = \frac{1}{|\mathcal{I}_{p,\tau}|} \sum_{j \in \mathcal{I}_{p,\tau}} v_j, \quad \mathcal{I}_{p,\tau} = \{j : p_j = p,\ \lfloor t_j \rfloor_{5\text{min}} = \tau\}.$$

For each pass, relative speed is:

$$\rho_j = \frac{v_j}{\overline{v}_{p_j, \lfloor t_j \rfloor_{5\text{min}}}},$$

and per driver:

$$\overline{\rho}_i = \frac{1}{|\mathcal{J}_i|} \sum_{j \in \mathcal{J}_i} \rho_j, \quad \mathcal{J}_i = \{j : \text{plate}_j = i\}.$$

1.3.3 Headway (time gap)

For each (p, ℓ) group (and optionally by month), records are ordered by (t, plate). The *headway* for observation j is computed as:

$$h_j = t_j - t_{j-1},$$

in seconds, where $j-1$ is the previous record within the same group. Headways greater than a threshold $h_{\max}$ (default 60s) are treated as missing. The mean *headway* per driver:

$$\overline{h}_i = \frac{1}{|\mathcal{H}_i|} \sum_{j \in \mathcal{H}_i} h_j, \quad \mathcal{H}_i = \{j : \text{plate}_j = i,\ h_j > 0,\ h_j \le h_{\max}\}.$$

1.3.4 TTC and conflict rate

Let $\Delta v_j = v_j - v_{j-1}$ be the speed difference between vehicle j and the preceding one in the same (p, ℓ). For closing situations ($\Delta v_j > 0$), the *Time-To-Collision* is defined as:

$$\text{TTC}_j = \frac{h_j\, v_j}{\Delta v_j}.$$

For each gantry p, a threshold τ_p (in code: `TTC_MAX_BY_PORTICO`) is defined. A conflict is counted if $\text{TTC}_j < \tau_{p_j}$, and the rate per driver is:

$$\text{conflict\_rate}_i = \frac{\sum_{j \in \mathcal{C}_i} \mathbf{1}[\text{TTC}_j < \tau_{p_j}]}{|\mathcal{C}_i|}, \quad \mathcal{C}_i = \{j : \text{plate}_j = i,\ h_j > 0\}.$$

In the implementation, to avoid extreme values, TTC_j is capped above at τ_{p_j}.

1.3.5 Lane changes

Ordering driver i's passes by time (and optionally by month), we define:

$$\text{lane\_changes}_i = \sum_{m=2}^{N_i} \mathbf{1}[\ell_{i,m} \neq \ell_{i,m-1}], \quad \text{lane\_change\_rate}_i = \frac{\text{lane\_changes}_i}{\max(N_i - 1, 1)}.$$

1.3.6 Monthly weighting (optional)

When monthly weighting is enabled, variables are computed per month and then consolidated per plate via a weighted average by $N_{i,m}$ (passes of driver i in month m):

$$\overline{x}_i = \frac{\sum_m N_{i,m}\, x_{i,m}}{\sum_m N_{i,m}}.$$

1.3.7 Temporal activity

To characterize how "persistent" a driver is over time, activity metrics are computed:

$$n_i^{(\text{days})} = \left|\left\{\lfloor t_j \rfloor_{\text{day}} :\ \text{plate}_j = i\right\}\right|, \quad n_i^{(\text{weeks})} = \left|\left\{\lfloor t_j \rfloor_{\text{week}} :\ \text{plate}_j = i\right\}\right|, \quad n_i^{(\text{months})} = \left|\left\{\lfloor t_j \rfloor_{\text{month}} :\ \text{plate}_j = i\right\}\right|.$$

1.4 Data preparation for clustering

A subset of numeric columns (*features*) is selected and rows with missing or infinite values in those columns are dropped. Before clustering, variables are standardized using *z-score*:

$$z = \frac{x - \mu}{\sigma},$$

using `sklearn.preprocessing.StandardScaler`.

1.5 Clustering methods

1.5.1 K-means

K-means seeks to partition the data into K clusters by minimizing the sum of within-cluster squared distances:

$$\min_{\{c_i\},\{\mu_k\}} \sum_{i=1}^{n} \|x_i - \mu_{c_i}\|^2 .$$

In the implementation, `sklearn.cluster.KMeans` is used and, for quick evaluation of K, optionally `MiniBatchKMeans`.

Selecting K (metrics) Metrics are computed for $K \in [K_{\min}, K_{\max}]$:

- **Silhouette**: $s(i) = \frac{b(i)-a(i)}{\max(a(i),b(i))}$.
- **Davies–Bouldin**: average ratio between within-cluster dispersion and between-cluster separation.
- **Calinski–Harabasz**: ratio between between-cluster and within-cluster dispersion.

1.5.2 Gaussian Mixture Models (GMM)

A GMM models the density as a mixture:

$$p(x) = \sum_{k=1}^{K} \pi_k \, \mathcal{N}(x \mid \mu_k, \Sigma_k),$$

typically estimated via EM. The selection of K is supported by information criteria:

$$\mathrm{AIC} = -2 \log L + 2p, \qquad \mathrm{BIC} = -2 \log L + p \log n,$$

where L is the likelihood, p the number of parameters, and n the number of samples.

1.5.3 HDBSCAN

HDBSCAN is a density-based method that builds a hierarchy of clusters and extracts stable groupings. Typical parameters:

- `min_cluster_size`: minimum cluster size.
- `min_samples`: controls robustness to noise.

Observations marked as noise receive label -1.

1.6 Frequent vs. rare drivers (optional)

To improve robustness when there are plates with few passes, the workflow can train the model on *frequent drivers* (e.g., $N_i \geq N_{\min}$) and then assign labels to rare drivers with a confidence threshold:

- **K-means**: compute the minimum distance to the nearest centroid; if it exceeds a percentile q estimated on the frequent set, label as unknown (-1).
- **GMM**: compute the maximum posterior probability; if it is below a threshold γ, label as unknown (-1).

A *confidence score* (`confidence_score`) and an `is_rare` indicator are also saved.

1.7 Outputs and artifacts

The module generates files in `Resultados/`:

- Per-plate variables in DuckDB: `cluster_features*.duckdb`.
- Per-plate labels: `cluster_kmeans_kK.csv`, `cluster_gmm_kK.csv`, `cluster_hdbscan.csv`.
- Per-cluster summary (*centroids*): `cluster_summary*.csv`.
- Descriptive statistics per cluster: `cluster_descriptive*.csv`.

1.8 Cluster variables aggregated by gantry and interval

In addition to plate-level clustering, gantry/interval aggregated variables are built from a label table (`plate` $\rightarrow$ `cluster_label`). For each gantry p, interval τ, and cluster c:

$$n_{p,\tau,c} = \#\{j : p_j = p,\ \lfloor t_j \rfloor_\Delta = \tau,\ \text{cluster}(\text{plate}_j) = c\}, \qquad \text{share}_{p,\tau,c} = \frac{n_{p,\tau,c}}{\sum_{c'} n_{p,\tau,c'}}.$$

Optionally, the following are aggregated:

- **Hourly flow**: $\text{flow}_{p,\tau,c} = n_{p,\tau,c} \cdot \frac{60}{\Delta} \cdot \frac{1}{L}$, where Δ is the interval size (minutes) and L is the assumed number of lanes.
- **Mean speed** per cluster: $\overline{v}_{p,\tau,c}$.
- **Density**: $\text{density}_{p,\tau,c} = \text{flow}_{p,\tau,c} / \overline{v}_{p,\tau,c}$ (with defensive handling when $\overline{v} \leq 0$).
- **Temporal variations**: Δspeed, Δdensity via differences between consecutive intervals.
- **Cluster-mixing entropy**: $H_{p,\tau} = -\sum_c \text{share}_{p,\tau,c} \log(\text{share}_{p,\tau,c})$.

2 Accident Prediction

2.1 Objective

The objective is to build and evaluate models that predict the occurrence of an accident at a gantry, using aggregated variables derived from vehicle detections (`flujos`) and, optionally, variables derived from driver *clusters*.

2.2 Data sources

Flows (detections) Each detection record contains, at minimum, a timestamp t, a gantry p, a vehicle category c, and a speed v. In the system, flows are stored in DuckDB and queried by time range.

Events (accidents) Accidents come from event files. They are filtered by type *Accident* and by *Roadway* (by default, expressway). Then a timestamp field `accidente_time` is constructed and the location (km, axis, carriageway) is mapped to nearby gantries using `Datos/Porticos.csv`. The assigned gantry is stored as `ultimo_portico`.

2.3 Temporal discretization and notation

Time is discretized into intervals of size Δ minutes. For an observation with timestamp t, we define the interval start as:

$$\tau(t) = \left\lfloor \frac{t}{\Delta} \right\rfloor \Delta.$$

Variables are aggregated by the pair (p, τ), where p is the gantry.

2.4 Feature engineering

2.4.1 Macro flow variables

For each gantry p, interval τ, and category c, we define:

$$n_{p,\tau,c} = \#\{\text{detections in } (p,\tau,c)\}, \qquad \overline{v}_{p,\tau,c} = \frac{1}{n_{p,\tau,c}} \sum v.$$

With L lanes for normalization, hourly flow and density are computed as:

$$q_{p,\tau,c} = \frac{60}{\Delta} \frac{n_{p,\tau,c}}{L}, \qquad k_{p,\tau,c} = \frac{q_{p,\tau,c}}{\overline{v}_{p,\tau,c}}.$$

Optionally, dynamic variables are added via first temporal differences per gantry:

$$\Delta\overline{v}_{p,\tau,c} = \overline{v}_{p,\tau,c} - \overline{v}_{p,\tau-\Delta,c}, \qquad \Delta k_{p,\tau,c} = k_{p,\tau,c} - k_{p,\tau-\Delta,c}.$$

Finally, a wide matrix is constructed with columns of the form `flow_<cat>`, `speed_<cat>`, `density_<cat>`, etc.

2.4.2 Aggregated cluster variables (optional)

If a label file by license plate exists (`plate` $\rightarrow$ `cluster_label`), detections are joined with their corresponding cluster and aggregated by (p, τ, κ), where κ is the cluster identifier:

$$n_{p,\tau,\kappa} = \#\{\text{detections in } (p,\tau,\kappa)\}, \qquad \text{share}_{p,\tau,\kappa} = \frac{n_{p,\tau,\kappa}}{\sum_{\kappa'} n_{p,\tau,\kappa'}}.$$

Optionally, flow, mean speed, density, and temporal differences are derived per cluster, along with a mixing measure:

$$H_{p,\tau} = -\sum_{\kappa} \text{share}_{p,\tau,\kappa} \log(\text{share}_{p,\tau,\kappa}).$$

These variables are stored as columns `cluster_share_*`, `cluster_flow_*`, `cluster_speed_*`, etc.

2.5 Target definition (label)

Let an accident have timestamp t_a and assigned gantry p_a. The system defines a one-interval *lead time*: the accident is labeled in the previous interval

$$\tau_a = \tau(t_a) - \Delta,$$

and the target is defined as:

$$y_{p,\tau} = \begin{cases} 1, & \text{if there exists an accident with } (p_a, \tau_a) = (p, \tau), \\ 0, & \text{otherwise.} \end{cases}$$

Thus, $y_{p,\tau} = 1$ represents "an accident will occur in the next interval."

2.6 Dataset construction

The final dataset contains, per row, a pair (p, τ) with numerical variables and `target`. Construction is:

1. Compute macro flow variables by (p, τ).
2. (Optional) Compute cluster variables by (p, τ) and join them by key `(portico, interval_start)`.
3. Add `target` by joining with the set of pairs (p_a, τ_a).

2.7 Temporal split (train/validation/test)

To avoid temporal *leakage*, a time-based split is performed using the `interval_start` column: timestamps are ordered and the final block is assigned to test (proportion `test_size`). In standard training, validation is also separated from the train/val block using `val_size`.

2.8 Imbalance handling

Accident occurrence is rare ($y = 1$). Two strategies are used:

- **Class weighting**: e.g., Random Forest with `class_weight="balanced"`.
- **SMOTE (optional)**: a balanced dataset is generated by applying SMOTE *only on the training set*. Synthetic rows are marked with `synthetic=True` and are not used for threshold validation.

2.9 Models

The following models are considered:

- **Random Forest**: tree ensemble with class weighting.
- **XGBoost**: gradient boosting with binary logistic objective.
- **SVM**: classifier with scaling (`StandardScaler`) and probabilistic output.

Each model produces a *score* $s(x) \in \mathbb{R}$ (probability or decision function).

2.10 Threshold calibration with FAR

The *false alarm rate* (FAR) and sensitivity are defined as:

$$\text{FAR} = \frac{FP}{FP + TN}, \qquad \text{Sens} = \frac{TP}{TP + FN}.$$

A threshold θ is selected on the validation set using the ROC curve. Given a target FAR $\leq \alpha$, we choose:

$$\theta = \arg \max_{\theta:\ \text{FAR}(\theta) \leq \alpha} \text{Sens}(\theta),$$

and then evaluate on test with $\hat{y} = \mathbf{1}[s(x) \geq \theta]$.

2.11 Evaluation metrics

On test, we report, among others:

$$\text{Accuracy},\ \text{Precision},\ \text{Recall},\ F1,\ \text{FAR},\ \text{Sens},\ \text{ROC-AUC},$$

as well as the confusion matrix.

2.12 Feature selection and optimization

Feature importance An importance ranking is computed with Random Forest (`feature_importances_`) to order variables.

Optuna + SMOTE Hyperparameters for the model and SMOTE are optimized. For each trial:

1. Apply SMOTE on train.
2. Train the model.
3. Calibrate the FAR-constrained threshold on validation.
4. Evaluate performance (e.g., F1) and select the best trial.

2.13 Experiments: effect of cluster variables

To demonstrate the effect of including clusters, two scenarios are compared:

- **Base**: top-K variables from the baseline ranking (no clusters).
- **Base+Cluster**: top-K variables from the combined ranking (with clusters).

This is repeated for $K = 5, 10, \ldots$ up to the maximum available, recording metrics per configuration.

2.14 Artifacts

The pipeline generates artifacts in `Resultados/` (names with timestamp/suffix):

- Flow features: `accident_flow_features_*.csv` and/or `.duckdb`.
- Cluster features: `accident_cluster_features_*.csv`.
- Balanced datasets: `accident_balanced_*.csv`.
- Optuna results: `optuna_*.json` and `optuna_*_trials.csv`.
- Iterative experiment results: `experiments_results_*.csv`.

3 Drift detection

3.1 Objective, scope, and relation to the paper

The module `src/drift_detection_app.py` implements a replication workspace for the paper *Evaluating recalibration strategies for real-time crash prediction: adaptive drift detection vs. cumulative retraining*. The file itself states that the full *Drift detection* section is intentionally hosted in a single Python source, and it concentrates methodology replication, feature construction, feature selection, experiment execution, and coverage verification.

The real scope of the module goes beyond a single drift detector. It includes:

- deterministic crash-prediction dataset preparation for the selected corridor;
- yearly batch recalibration strategies;
- adaptive strategies based on ADWIN, ARF, and KSWIN;
- persistence for checkpoints, tuning caches, and SMOTE artifacts;
- reporting layers aligned with Appendix A.6–A.9 and Figure 7 of the paper.

The reference paper setup is fixed through `PAPER_TIME_SPAN = 2018-01-01 to 2024-09-30`. In addition, the focus corridor is constrained to the fixed gantry set `11, 12, 14, 15` (`DRIFT_FOCUS_PORTICOS`), while the segment order used by the article feature engineering follows `15→14`, `14→12`, and `12→11`.

3.2 Dependencies and data sources

The module assumes `Datos/flujos.duckdb` as the main flow source and `flujos_duckdb` as the input table. For events it uses CSV files under `Datos/`, processed with `process_accidentes_df()`, and it relies on `Datos/Porticos.csv` for geospatial mapping and gantry relationships via `load_porticos()`, `find_candidate_porticos()`, and `get_portico_segments()`.

The app also depends on optional libraries depending on the stage:

- `duckdb` for feature querying and persistence;
- `optuna` for hyperparameter tuning;
- `imblearn` for `SMOTE`;
- `river` for `ADWIN`, `KSWIN`, and `AdaptiveRandomForestClassifier`.

3.3 Tab-by-tab UI map

The `main()` function creates six tabs with the actual labels `Blueprint`, `Eventos`, `Feature engineering`, `Feature Selection`, `Experiments`, and `Coverage`. Each tab corresponds to an explicit functional block:

Blueprint `_render_blueprint_tab()` shows the article replication matrix, the related-work table (`build_related_work_table()`), covered sections/figures/tables, and the migration plan from R to Python (`build_python_migration_review_plan()`).

Eventos `_render_events_tab()` loads one or multiple accident files from `Datos/`, assigns them to gantries with `process_accidentes_df()`, preserves excluded events when no mapping is possible, and builds a corridor-level summary that mirrors *Crash prediction*.

Feature engineering `_render_feature_engineering_tab()` offers three sources: load an existing DuckDB, recompute features, or reuse the in-memory dataset. In computation mode, it queries `flujos.duckdb`, generates gantry/segment/lane variables, builds `target`, applies Stage 1 and Stage 2, and exports a DuckDB containing `raw_features` and `clean_features`.

Feature Selection `_render_feature_selection_tab()` runs correlation filtering, Random-Forest ranking, and persistence of the final feature-selection payload inside the same feature DuckDB file.

Experiments `_render_experiments_tab()` locks the final feature set, allows the user to select batch models, strategies, ARF/KSWIN variants, balance modes, and sequential seeds, and then invokes `run_recalibration_experiments()` with support for checkpoint preview, resume, and fresh-start execution.

Coverage `_render_coverage_tab()` validates that all sections, figures, and tables declared in `ARTICLE_SECTIONS`, `ARTICLE_FIGURES`, and `ARTICLE_TABLES` map to concrete Python artifacts. This is summarized through `article_coverage_percentage()`.

3.4 Feature engineering and target construction

Feature engineering is centered on `_compute_drift_article_features()`, supported by `feature_catalog_table()` and `feature_engineering_reference_table()`. The goal is to replicate Table 4 of the paper and produce a prediction-ready dataset on a 5-minute time grid while preserving the exact operational semantics of the code.

Operational notation Let $\Delta = 5$ minutes denote the interval width, and define

$$b(\tau) = \left\lfloor \frac{\tau}{\Delta} \right\rfloor \Delta$$

as the function that maps a timestamp τ to the start of its interval. Under the default article configuration, the corridor uses the gantry order

$$P = (15, 14, 12, 11),$$

the category set

$$C = \{\text{Light}, \text{Heavy}, \text{Motorcycle}\},$$

and the consecutive segments

$$S = \{(15, 14), (14, 12), (12, 11)\}.$$

Each raw AVI record contributes a timestamp τ_r, point speed v_r, category c_r, license plate π_r, gantry p_r, and lane ℓ_r. The app maps `CATEGORIA` to `Light/Heavy/Motorcycle` through `DRIFT_ARTICLE_CATEGORY_MAP`, namely $\{1 \mapsto \text{Light}, 2 \mapsto \text{Heavy}, 3 \mapsto \text{Heavy}, 4 \mapsto \text{Motorcycle}\}$.

Generated feature space With the fixed article corridor of four gantries, three segments, three categories, and three lanes, the default configuration yields 260 numeric predictors before adding `target`:

- 96 gantry-by-category variables: 4 base metrics + 4 deltas, for 4 gantries and 3 categories;
- 8 compositional gantry variables: `ft_motorcycle_*` and `ft_heavy_*`;
- 72 segment-by-category variables: 5 base metrics + 3 deltas, for 3 segments and 3 categories;
- 21 segment-global variables: 4 base metrics + 3 deltas, for 3 segments;
- 63 origin-lane variables: 4 base metrics + 3 deltas, for 3 segments and 3 lanes.

If the user restricts gantries or categories in the UI, this space shrinks, but the mathematical definitions remain unchanged.

Gantry-by-category variables For each interval t, gantry $p \in P$, and category $c \in C$, define the observation set

$$G_{t,p,c} = \{r : b(\tau_r) = t,\ p_r = p,\ c_r = c\}.$$

The code then builds the `flow_`, `vel_`, `sd_`, and `den_` columns as

$$\mathrm{Flow}_{t,p,c} = |G_{t,p,c}|,$$

$$\mathrm{Vel}_{t,p,c} = \frac{1}{|G_{t,p,c}|} \sum_{r \in G_{t,p,c}} v_r,$$

$$\mathrm{Sd}_{t,p,c} = \sqrt{\frac{1}{|G_{t,p,c}| - 1} \sum_{r \in G_{t,p,c}} (v_r - \mathrm{Vel}_{t,p,c})^2},$$

$$\mathrm{Den}_{t,p,c} = \begin{cases} \dfrac{\mathrm{Flow}_{t,p,c}}{\mathrm{Vel}_{t,p,c}}, & \text{if } \mathrm{Vel}_{t,p,c} > 0, \\ \mathrm{NA}, & \text{otherwise.} \end{cases}$$

In the actual implementation, Flow is filled with 0 when no observations exist, whereas Vel, Sd, and Den remain missing when the group does not exist or lacks enough support. In addition, Sd uses the sample standard deviation from `pandas`; therefore, when only one vehicle is present, the value remains `NaN`.

Temporal changes are built through first differences inside each time series:

$$\Delta X_{t,p,c} = X_{t,p,c} - X_{t-\Delta,p,c}, \qquad X \in \{\mathrm{Flow}, \mathrm{Vel}, \mathrm{Den}, \mathrm{Sd}\}.$$

This produces `delta_flow_*`, `delta_vel_*`, `delta_den_*`, and `delta_sd_*`. For the first interval where the base variable is defined, the code sets the delta to 0; if the base variable starts as missing, the delta also remains missing.

Compositional gantry shares At the same interval and gantry, the app computes the total flow

$$\text{Flow}^{\text{tot}}_{t,p} = \text{Flow}_{t,p,\text{Light}} + \text{Flow}_{t,p,\text{Heavy}} + \text{Flow}_{t,p,\text{Motorcycle}}.$$

It then derives the shares

$$\text{FtMotorcycle}_{t,p} = \frac{\text{Flow}_{t,p,\text{Motorcycle}}}{\text{Flow}^{\text{tot}}_{t,p}}, \qquad \text{FtHeavy}_{t,p} = \frac{\text{Flow}_{t,p,\text{Heavy}}}{\text{Flow}^{\text{tot}}_{t,p}}.$$

These are the `ft_motorcycle_*` and `ft_heavy_*` columns. If the total gantry flow is zero, both shares remain `NaN` instead of being forced to zero, because the denominator does not represent an observed mixture.

Segment matching between consecutive gantries Segment features are not computed from local gantry speeds, but from matched trips between consecutive gantries. For each segment $s = (p^-, p^+) \in S$, the distance d_s is obtained through `_lookup_article_segment_distance_km()`, prioritizing roadway metadata and falling back to geometric estimates when needed.

The maximum matching window depends on segment distance:

$$\tau_s = \max\left(30,\ \frac{60\, d_s}{20}\right) \text{ minutes},$$

because `DRIFT_ARTICLE_MIN_MATCH_WINDOW_MINUTES = 30` and the minimum reference speed is 20 km/h. An upstream-downstream pair is accepted if:

- it shares the same plate π;
- the downstream passage occurs after the upstream passage and within τ_s;
- the category is consistent at both ends;
- travel time is strictly positive.

The implementation uses `merge_asof(..., direction="backward")` from the downstream gantry back to the upstream one, then removes duplicates by `plate_clean` and `time_start`, keeping the first compatible downstream passage for each upstream observation. The match interval is anchored to the upstream timestamp, that is, to $b(\tau_{\text{start}})$.

For each matched trip m on segment s, define the travel time

$$\text{TT}_m = \tau^{\text{end}}_m - \tau^{\text{start}}_m,$$

and the spatial speed

$$\text{SegVel}_m = \begin{cases} \dfrac{d_s}{\text{TT}_m / 1\text{ hour}}, & \text{if } d_s > 0, \\ \text{NA}, & \text{if no valid distance is available.} \end{cases}$$

In parallel, the lane-change indicator is

$$\text{LC}_m = \mathbf{1}\{\ell^{\text{start}}_m \neq \ell^{\text{end}}_m\},$$

whenever both lanes are known; otherwise the match contributes `NaN` to that component.

Segment-by-category variables Let $M_{t,s,c}$ denote the set of matches for interval t, segment s, and category c. From that set the code builds columns such as `flow_light_15_14`, `vel_heavy_14_12`, or `cl_motorcycle_12_11`:

$$\text{Flow}_{t,s,c} = |M_{t,s,c}|,$$

$$\text{Vel}_{t,s,c} = \frac{1}{|M_{t,s,c}|} \sum_{m \in M_{t,s,c}} \text{SegVel}_m,$$

$$\text{Sd}_{t,s,c} = \sqrt{\frac{1}{|M_{t,s,c}| - 1} \sum_{m \in M_{t,s,c}} (\text{SegVel}_m - \text{Vel}_{t,s,c})^2},$$

$$\text{CL}_{t,s,c} = \frac{1}{|M^{LC}_{t,s,c}|} \sum_{m \in M^{LC}_{t,s,c}} \text{LC}_m,$$

$$\text{Den}_{t,s,c} = \begin{cases} \dfrac{\text{Flow}_{t,s,c}}{\text{Vel}_{t,s,c}}, & \text{if } \text{Vel}_{t,s,c} > 0, \\ \text{NA}, & \text{otherwise.} \end{cases}$$

Here $M^{LC}_{t,s,c}$ denotes the subset with observed lanes at both ends. The deltas ΔFlow, ΔVel, and ΔDen reuse the same first-difference definition as the gantry block. There is no ΔCL in the current implementation.

Segment-global and origin-lane variables The segment-global block aggregates all matches in the interval without splitting by category:

$$M_{t,s} = \bigcup_{c \in C} M_{t,s,c}.$$

On top of $M_{t,s}$ the code generates `flow_15_14`, `vel_15_14`, `sd_15_14`, `den_15_14`, and their temporal deltas. Mathematically, the formulas are identical to the segment-by-category block, except that $M_{t,s,c}$ is replaced by $M_{t,s}$.

To capture lane heterogeneity, the same matching output is re-aggregated by origin lane:

$$M_{t,s,\ell} = \{m \in M_{t,s} : \ell^{\text{start}}_m = \ell\}, \qquad \ell \in \{1, 2, 3\}.$$

The app then computes $\text{Flow}_{t,s,\ell}$, $\text{Vel}_{t,s,\ell}$, $\text{Sd}_{t,s,\ell}$, $\text{Den}_{t,s,\ell}$, and their deltas, producing columns such as `vel_lane2_15_14` or `delta_den_lane3_12_11`. This block matters because the code explicitly motivates it by the empirical relevance of lane-specific variables in the Figure 6 replica.

Boundary rules and numerical consistency The implementation distinguishes between absence of traffic and absence of measurement:

- flows are filled with 0 when the group has no observations;
- speeds, standard deviations, densities, and lane-change fractions remain `NaN` when there is not enough information;
- deltas are computed on the already pivoted feature matrices, so they inherit the same missing-value pattern;
- when computation is done in DuckDB batches, static features match the one-pass computation, and deltas are reinitialized at batch boundaries with 0 or `NaN`; this is covered by `test_drift_batched_flow_featu`

Target construction The target is added by `_add_interval_accident_target()` after corridor accidents are filtered with `_filter_accidents_for_allowed_porticos()`. If t_a is an accident timestamp, its binary label is assigned to the interval

$$t^\star = b(t_a) - \Delta.$$

Therefore,

$$y_t = 1 \iff \exists a \text{ such that } b(t_a) = t + \Delta.$$

This definition implements a one-bin lead time: features at t aim to anticipate accidents occurring in the next 5-minute interval. The test `test_drift_article_features_cover_table4_and_target_shift` validates this shift explicitly; for example, an accident at 00:06 activates `target = 1` on the 00:00 row, not on the 00:05 row.

3.5 Data preparation: Stage 1 and Stage 2

Paper Section 4.2 is operationalized through `apply_missing_data_policy()`, `identify_long_zero_accident_runs`, `filter_long_zero_accident_runs()`, and `run_configurable_preparation_pipeline()`.

- **Stage 1**: remove variables with more than 1% missing values, then drop intervals with any missing predictor among the remaining variables.
- **Stage 2**: remove long accident-free windows, by default at least 7 consecutive days with `target = 0`.

The output includes a `DataPreparationSummary` with counts for initial/final rows, removed variables, detected windows, and discarded rows. The test `test_configurable_preparation_pipeline_respects_sta` validates that both stages respond correctly to their UI toggles.

3.6 Feature selection

The selection stage uses `compute_abs_correlations()`, `drop_highly_correlated_features()`, and `rank_features_random_forest()`. Operationally, it runs in two steps:

1. remove highly correlated variables using a configurable threshold;
2. train a Random Forest to rank the remaining variables and keep the effective top-N for experiments.

The same tab generates the Figure 5 replica (density of $|\rho|$) and the Figure 6 replica (top feature importances), and it persists the selection payload inside the feature DuckDB. The test `test_feature_selection_payload_persists_with_feature_duckdb` covers this persistence path.

3.7 Experimental design and strategies

Available strategies are defined by `EXPERIMENT_STRATEGIES`: `static`, `period_aligned`, `cumulative`, `adaptive_adwin`, `adaptive_arf`, and `adaptive_kswin`. Batch models supported through `MODEL_NAMES` are `XGBoost`, `AdaBoost`, `Random Forest`, and `NNet`; online variants are `ARFmoderate`, `ARFmaj`, `ARFfast`, `KSWINpaper`, and `KSWINseasonal`.

Formal experimental setup Let $Y_0 < Y_1 < \cdots < Y_J$ denote the sequence of years observed in the dataset once it is sorted by `interval_start`. The app uses Y_0 as the default base year unless the user overrides it. For each interval t, the pair (x_t, y_t) is formed by the selected feature vector and the one-step-ahead accident target. The experiment always runs on the dataset already cleaned by Stage 1 and Stage 2.

The main orchestrator `run_recalibration_experiments()` builds experiment blocks over the Cartesian product of:

- strategies;
- batch models or online variants;
- balance modes `none`/`smote` whenever applicable;
- sequential repetition seeds.

Therefore, the final output is not a single run but a collection of persisted blocks that are assembled at the end. Formally, if B is the set of experiment blocks and R the set of seeds, the base number of executions is $|B| \times |R|$, plus the canonical global tuning tasks.

Internal batch training and threshold calibration Batch strategies, `adaptive_adwin`, and `adaptive_kswin` all reuse `train_model_with_internal_validation()`. Given a training split T, the actual code executes:

1. a stratified internal split $(T^{\text{fit}}, T^{\text{val}})$ with validation proportion $\rho = 0.2$ by default;
2. hyperparameter tuning on T^{fit} with stratified cross-validated AUC;
3. threshold calibration on the full T using out-of-fold scores;
4. final refit on all rows in T, with optional SMOTE.

Let $\Theta_{m,b}$ be the search space associated with model m and balance mode b. Hyperparameter selection solves

$$\hat{\theta} = \arg\max_{\theta \in \Theta_{m,b}} \frac{1}{K} \sum_{k=1}^{K} \text{AUC}_k(\theta),$$

where K is the effective number of stratified folds. If `balance_mode = smote`, the search space itself includes `sampling_strategy` and `k_neighbors`; if `balance_mode = none`, only model hyperparameters are tuned.

Once $\hat{\theta}$ is fixed, the code generates out-of-fold scores s_i^{cv} for every row $i \in T$ and selects the operating threshold through the Youden criterion:

$$\hat{\tau} = \arg\max_{\tau} \left[\text{TPR}(\tau) - \text{FPR}(\tau)\right] = \arg\max_{\tau} \left[\text{Sensitivity}(\tau) + \text{Specificity}(\tau) - 1\right].$$

Hard predictions are defined by

$$\hat{y}_i = \mathbf{1}\{s_i \geq \hat{\tau}\}.$$

If out-of-fold calibration becomes ill-posed, the code falls back to the internal holdout T^{val}.

Balancing with SMOTE The `smote` mode is never applied to the test year or to the evaluation stream; it is restricted to training folds and the final refit set. If the minority class has fewer than two observations, or if the minority/majority ratio is already above the requested target, balancing is skipped. Otherwise, SMOTE builds a balanced set $(X^{\text{bal}}, y^{\text{bal}})$ satisfying

$$\frac{N_{\text{minor}}^{\text{new}}}{N_{\text{major}}} \approx \texttt{sampling\_strategy},$$

with an effective neighborhood

$$k_{\text{eff}} = \min(k_{\text{neighbors}}, N_{\text{minor}} - 1).$$

SMOTE artifacts may be persisted and reused at block level through `_load_or_create_smote_artifact()`.

Yearly batch strategies `run_yearly_strategy()` implements three training rules for each target year Y_j, $j \geq 1$:

$$\mathcal{T}_j^{\text{static}} = \{t : \text{year}(t) = Y_0\},$$
$$\mathcal{T}_j^{\text{period}} = \{t : \text{year}(t) = Y_j - 1\},$$
$$\mathcal{T}_j^{\text{cum}} = \{t : \text{year}(t) \leq Y_j - 1\},$$

while the evaluation set is always

$$\mathcal{E}_j = \{t : \text{year}(t) = Y_j\}.$$

Each output row contains `training_year`, `prediction_year`, `model`, `balance_mode`, performance metrics, the calibrated threshold, training time, and `n_train`/`n_test`. In the `static` strategy, because $\mathcal{T}_j^{\text{static}}$ is identical for all future years, the implementation caches and reuses the fitted bundle to avoid redundant tuning and refitting; this behavior is validated by `test_run_yearly_strategy_reuses_static_tra`

The expected result of this block is:

- one row in `yearly_results` per (strategy, model, balance, Y_j, seed);
- one ROC payload segment per prediction year in `roc_payload`;
- Appendix Tables A.6, A.7, and A.8 for `static`, `period_aligned`, and `cumulative`, respectively.

Adaptive recalibration with ADWIN `run_adaptive_strategy()` first trains a batch model on the base year Y_0, then processes the post-base stream chronologically. For each stream row it computes a score s_t, a hard prediction $\hat{y}_t = \mathbf{1}\{s_t \geq \hat{\tau}\}$, and a binary error signal

$$e_t = \mathbf{1}\{\hat{y}_t \neq y_t\}.$$

The `SimpleADWIN` class maintains an error window W. For each candidate split $W = W_0 \cup W_1$, with sizes n_0 and n_1, it defines

$$\mu_0 = \frac{1}{n_0} \sum_{e \in W_0} e, \qquad \mu_1 = \frac{1}{n_1} \sum_{e \in W_1} e,$$

$$m = \left(\frac{1}{n_0} + \frac{1}{n_1}\right)^{-1}, \qquad \epsilon = \sqrt{\frac{1}{2m} \log\left(\max\left(1.0000001, \frac{4 \log n}{\delta}\right)\right)},$$

where $n = |W|$ and $\delta =$ `adwin_delta`. Drift is declared when

$$|\mu_0 - \mu_1| > \epsilon.$$

When that happens, the code:

- closes the accumulated prediction segment up to t;
- stores `drift_date`, `W`, `W0`, `W1`, and `remaining_periods`;
- attempts recalibration using the last $|W_1|$ stream rows.

Retraining only happens if the W_1 window contains at least $\max(20, \lfloor \texttt{min\_window}/10 \rfloor)$ rows, or the user-specified `min_retrain_size`, and both target classes are present. Otherwise, the segment is still closed, but the current model continues unchanged. The expected result is an `adaptive_results` table with one row per detected drift plus one final segment with `remaining_periods = 0`; those rows feed the ADWIN portion of Appendix A.9.

Adaptive Random Forest `run_arf_strategy()` implements a genuinely online strategy. On the base year Y_0, `train_arf_with_internal_validation()` separates a trailing chronological validation block and uses the leading block to warm up the ARF ensemble. It then predicts sequentially on the validation block, learns each label immediately, and fixes $\hat{\tau}$ through the same Youden criterion.

During the subsequent stream, the model is updated one observation at a time:

$$s_t = f_{t-1}(x_t), \qquad f_t = \mathcal{U}(f_{t-1}, x_t, y_t),$$

where $\mathcal{U}$ is the online `learn_one` update. Unlike ADWIN and KSWIN, there is no external batch retraining after drift: adaptation is internal to the forest. For that reason, the code never duplicates this strategy by `balance_mode`; it always uses `not_applicable`.

Results are segmented by prediction year rather than by externally detected drift. Each expected row contains:

- `prediction_year`;
- `segment_rows`;
- `n_internal_drifts`;
- `n_internal_warnings`;
- performance metrics and the operating threshold.

As a consequence, Appendix A.9 represents ARF as yearly online segments rather than as $W/W_0/W_1$ windows. The tests `test_run_arf_strategy_variants` and `test_recalibration_experiments_support_adaptiv` validate this output shape.

Adaptive recalibration with KSWIN `run_kswin_strategy()` clearly separates covariate-drift detection from prediction. It first trains a batch model on Y_0. It then selects monitored variables with `_select_kswin_monitor_columns()`, which in the current implementation simply takes the first `top-k` numeric columns from the ordered `feature_cols` list. This is not an online ranking: it is a deterministic choice on top of the already approved feature set.

For each monitored feature f, KSWIN operates in one of two spaces:

- `KSWINpaper`: the raw value $x_{t,f}$;
- `KSWINseasonal`: a seasonal z-score

$$z_{t,f} = \frac{x_{t,f} - \mu_{d(t),h(t),f}}{\sigma_{d(t),h(t),f}},$$

with fallback to hour-level profiles and then to global base-year profiles.

Each detector uses $\alpha = 10^{-4}$, `window_size` $= 300$, and `stat_size` $= 30$, so Appendix A.9 reports

$$W = 300, \qquad W_0 = 270, \qquad W_1 = 30$$

as fixed detector sizes rather than learned adaptive windows.

Let F_t be the set of features whose detector fires at time t. The block-level drift condition is:

$$|F_t| \geq \texttt{vote\_threshold}.$$

When that happens, the prediction segment is closed, `vote_count`, `detected_features`, and `monitored_features` are stored, and a temporal retraining window is selected:

$$\mathcal{R}_t = \{i : t_i \geq t - D\},$$

where $D =$ `kswin_retrain_days`. If $|\mathcal{R}_t| <$ `kswin_min_retrain_rows`, the code uses the last `min_rows` observations; if that window still contains a single target class, it expands to all observations seen up to t. After batch refitting, the KSWIN detectors are rebuilt and rewarmed on the retraining window.

The expected result of KSWIN is one row per voted drift plus a final segment, with extra columns such as `vote_threshold`, `monitor_feature_count`, `retrain_rows`, and `retrain_positive_rows`. The tests `test_run_kswin_strategy_variants`, `test_recalibration_experiments_support_adaptive_kswin`, and `test_kswin_optuna_studies_capture_base_and_retrains_with_balance_mode` validate that behavior.

Seeds, global tuning, persistence, and resume `run_recalibration_experiments()` does not tune every batch block from scratch. Before launching blocks, it constructs canonical tuning tasks on the base-year split Y_0 for each (model, balance) combination that will be reused by yearly strategies, ADWIN, or KSWIN. That global tuning is persisted in `tuning_dir` and later injected as `fixed_params`/`fixed_tuning_artifact` into downstream blocks. Methodologically, this enforces comparability across strategies that share the same model family, because they all start from the same canonical $\hat{\theta}$ per seed and balance mode.

Each block persists:

- yearly or adaptive result rows;
- segmented ROC payload;
- its own `execution_log`;
- references to tuning and SMOTE artifacts.

The central `manifest.json` maintains progress, completed blocks, available caches, and error status. If a run fails midway, the next execution can resume from the checkpoint and reuse already computed tuning/SMOTE artifacts; this is covered by `test_preview_recalibration_checkpoint_detects_resumable_mani` and `test_recalibration_experiments_resume_from_failed_checkpoint`.

Expected results Methodologically, a completed experiment must produce seven mutually consistent outputs:

- `yearly_results`: one row per yearly batch split, with `strategy`, `prediction_year`, `model`, `balance_mode`, metrics, threshold, and sample sizes;

- `adaptive_results`: one row per adaptive segment, either drift-delimited for ADWIN/KSWIN or year-delimited for ARF;
- `summary`: mean metrics by (`strategy`, `model`, `balance_mode`), including `n_segments` and `n_repetitions`;
- `appendix_tables`: A.6 for `static`, A.7 for `period_aligned`, A.8 for `cumulative`, and A.9 for adaptive results;
- `appendix_tables_mean`: the same tables averaged across sequential seeds, enriched with `seed_list` and `run_orders`;
- `average_roc`: Figure 7 style mean ROC curves, built by interpolating each individual curve on a uniform 101-point FPR grid and averaging TPR:

$$\overline{\mathrm{TPR}}_g(u) = \frac{1}{J_g} \sum_{j=1}^{J_g} \widetilde{\mathrm{TPR}}_{g,j}(u);$$

- `run_manifest` and `execution_log`: reproducible traceability for configuration, seeds, thresholds, tuning, checkpoints, failures, and resource consumption.

The tests `test_end_to_end_recalibration_and_average_roc`, `test_recalibration_experiments_compare_bal` and `test_recalibration_experiments_persist_optuna_json` verify precisely that this result package exists, is internally consistent, and can be persisted as the final JSON artifact.

3.8 Metrics, tables, and final artifacts

Evaluation uses four base metrics computed by `compute_classification_metrics()`: `auc`, `sensitivity`, `specificity`, and `error_rate`, plus the operational `threshold`. On top of those tables, the module builds:

- consolidated summaries with `summarize_results()`;
- Appendix A.6–A.9 tables with `format_appendix_tables()`;
- seed-averaged appendix tables with `format_appendix_tables_mean()`;
- Figure 7 style average ROC curves with `build_average_roc_curves()`.

Primary persistence is organized in three layers:

- exported feature DuckDB files under `Resultados/`;
- per-run checkpoints under `Resultados/drift_recalibration_runs/`, containing `manifest.json`, persisted blocks, tuning cache, and SMOTE artifacts;
- the final results JSON written by `_persist_drift_recalibration_json()` as `drift_recalibration_optuna_*.`

The `run_manifest` stores configuration, seeds, strategies, grid sizes, active features, and checkpoint paths. The `execution_log` captures tuning steps, training events, errors, adaptive retraining, and resource traces.

3.9 Validation through tests

The file `tests/test_drift_detection_app.py` provides broad functional coverage of the pipeline. The most relevant cases include:

- `test_article_coverage_is_100_percent`;
- `test_drift_article_features_cover_table4_and_target_shift`;
- `test_drift_batched_flow_features_duckdb_consistency`;
- `test_end_to_end_recalibration_and_average_roc`;
- `test_recalibration_experiments_compare_balance_modes_end_to_end`;
- `test_recalibration_experiments_persist_optuna_json`;
- `test_recalibration_experiments_resume_from_failed_checkpoint`;
- `test_recalibration_experiments_support_adaptive_arf`;
- `test_recalibration_experiments_support_adaptive_kswin`.

Taken together, these tests show that the *Drift detection* documentation describes repository behavior that already exists, rather than an aspirational design.

4 Introduction

This document details the *feature engineering* process used to cluster drivers (license plates) based on their behavior on the highway. The goal is to transform raw vehicle flow records into a single feature vector per license plate.

5 Data Preprocessing

Before computing the variables, the flow data (F) go through a cleaning stage:

1. **License Plate Normalization:** Non-alphanumeric characters are removed and the plate is standardized to uppercase. Invalid plates (e.g., "NULL", "NAN") or plates with length outside the range $[5, 6]$ are discarded.
2. **Lane Filtering:** Only lanes 1, 2, and 3 are considered.
3. **Duplicate Removal:** Duplicate records are removed considering the tuple $(Plate, Gantry, Lane, Timestamp$
4. **Outlier Handling (Speed):** *Winsorization* is applied to speed for each $(Gantry, Lane)$ group, capping extreme values at the 1% and 99% percentiles.

6 Variable Computation (Event Level)

For each event i (a vehicle passing through a gantry), intermediate metrics are computed. Let v_i be the speed of event i at time t_i.

6.1 Headway (h_i)

Headway is computed as the time difference with the preceding vehicle $(i-1)$ in the **same lane and gantry**. The data must be temporally ordered.

$$h_i = t_i - t_{i-1} \tag{1}$$

If $h_i > 60$ seconds, it is assumed there is no close interaction and the value is discarded (NaN) for conflict computation purposes, although it may be included in averages if desired.

6.2 Indexed Relative Speed ($v_{rel,i}$)

To normalize behavior according to traffic conditions, the vehicle speed is compared with the average flow speed at that gantry over a 5-minute interval, denoted $\bar{V}_{5min}$.

$$v_{rel,i} = \frac{v_i}{\bar{V}_{5min}} \tag{2}$$

- $v_{rel,i} > 1$: Driver is faster than the average flow.
- $v_{rel,i} < 1$: Driver is slower than the average flow.

6.3 Conflict and TTC (Time To Collision)

Collision risk with the preceding vehicle is evaluated. TTC_i is computed only if vehicle i is traveling faster than the previous vehicle ($v_i > v_{i-1}$) and the headway is valid.

$$TTC_i = \frac{h_i \cdot v_i}{v_i - v_{i-1}} \tag{3}$$

A **conflict** event (c_i) is defined if TTC_i is below a gantry-specific threshold (TTC_{max}), defined empirically.

$$c_i = \begin{cases} 1 & \text{if } TTC_i < TTC_{max} \\ 0 & \text{if } TTC_i \geq TTC_{max} \text{ or } v_i \leq v_{i-1} \end{cases} \tag{4}$$

7 Aggregation by License Plate

The goal is to obtain a single vector per driver P. The aggregation process can be performed in two ways: Global or Monthly-Weighted.

Let N_P be the total number of recorded passes for license plate P.

7.1 Aggregated Variables

7.1.1 Average Speed ($\bar{v}_P$)

$$\bar{v}_P = \frac{1}{N_P} \sum_{i \in P} v_i \tag{5}$$

7.1.2 Average Relative Speed ($\bar{v}_{rel,P}$)

$$\bar{v}_{rel,P} = \frac{1}{N_{rel}} \sum_{i \in P} v_{rel,i} \tag{6}$$

Where N_{rel} is the number of events with valid relative speed.

7.1.3 Average Headway ($\bar{h}_P$)

$$\bar{h}_P = \frac{1}{N_h} \sum_{i \in P, h_i \leq 60} h_i \tag{7}$$

7.1.4 Conflict Rate (CR_P)

Proportion of events with valid headway that resulted in a conflict.

$$CR_P = \frac{\sum_{i \in P} c_i}{N_h} \tag{8}$$

7.1.5 Lane Usage ($Lane_k$)

Proportion of usage of lane $k \in \{1, 2, 3\}$.

$$Lane_k = \frac{\text{Count of passes in lane } k}{N_P} \tag{9}$$

7.1.6 Lane Change Rate (LCR_P)

Estimated by counting how many times the vehicle changed lanes between consecutive gantries. Let L_j be the lane in event j (time-ordered).

$$Changes = \sum_{j=2}^{N_P} \mathbb{I}(L_j \neq L_{j-1}) \tag{10}$$

$$LCR_P = \frac{Changes}{N_P - 1} \tag{11}$$

Note: This metric is approximate, as it assumes continuity between recorded gantries.

7.2 Consolidation Strategies

7.2.1 1. Global Aggregation (Default)

The formulas above are applied over **all** historical records for license plate P as a single set.

7.2.2 2. Monthly Weighting

Monthly averages $X_{P,m}$ are computed first for each month m, and then weighted by the number of trips in that month ($N_{P,m}$).

Let X be any variable (e.g., Speed, Conflict Rate). The final weighted value is:

$$X_{P,final} = \frac{\sum_m (X_{P,m} \cdot N_{P,m})}{\sum_m N_{P,m}} \tag{12}$$

This ensures that months with higher activity have greater influence, while still allowing intermediate monthly metrics if temporal analysis is required.

8 Definition of Frequent Drivers

For clustering model training, users are separated into:

- **Frequent:** Users with enough history to characterize their behavior. Typical criteria:
  - $N_P \geq 20$ total passes.
  - Active days ≥ 5.
- **Infrequent:** Users with few data points. They are not used for training, but can be assigned to a cluster later using the trained model (or remain as "Unknown").

9 Python Implementation

The computation logic described above is implemented mainly in the module `src/clustering.py`. Below, we detail the execution flow of the key functions for auditing purposes.

9.1 Data Cleaning: `clean_flujos_for_clustering`

This function takes the raw flow DataFrame and performs the initial preparation:

1. **Column Validation:** Ensures that essential columns exist (timestamp, speed, gantry, lane, license plate).
2. **License Plate Cleaning:** Plates are normalized (alphanumeric, uppercase) and filtered if their length is invalid (outside the range $[5, 6]$).
3. **Basic Filtering:** Rows with null values in critical fields are removed and only lanes 1, 2, and 3 are kept.
4. **Deduplication:** Exact duplicates of the tuple (Plate, Gantry, Lane, Timestamp) are removed. If duplicates exist with different speeds, a deterministic priority is applied (merge sort ordering preserving the first).
5. **Winsorization:** Extreme speed values are capped by group (Gantry, Lane) using the defined percentiles (default 1% and 99%).

9.2 Feature Computation: `Clusterization`

This is the main function that orchestrates feature computation.

9.2.1 1. Simple Metrics and Pre-computation

Metrics that do not require complex temporal ordering are computed:

- **Total Passes (`total_passes`):** Simple count of records per license plate.
- **Average Speed (`avg_speed_kmh`):** Arithmetic mean of speed.
- **Lane Usage:** A pivot table is built to count passes per lane and divide by the total.

9.2.2 2. Relative Speed

- The average flow speed is computed in 5-minute windows per gantry (`interval_speed_mean`).
- This average is joined to each event to compute `relative_speed` $= v_i / \bar{V}_{5min}$.

9.2.3 3. Interaction Metrics (Headway, Conflict)

This stage is critical and sensitive to data ordering:

1. **Sorting:** Data are strictly sorted by Gantry, Lane, and Timestamp.
2. **Vectorized Operations (Shift):** Shifted columns (`shift(1)`) are created to compare the current event with the immediately preceding vehicle in the same lane/gantry.
3. **Headway Computation:** Time difference between the current and previous event. Headways greater than `max_headway_s` (60s) are discarded (NaN).
4. **Conflict Computation:**
   - Compute $TTC = \frac{Headway \cdot v_{current}}{v_{current} - v_{previous}}$.
   - Valid only if $v_{current} > v_{previous}$.
   - Mark conflict if $TTC < TTC_{max}$ (where TTC_{max} depends on the gantry).
5. **Aggregation:** Valid events are summed and counted by license plate to obtain averages.

9.2.4 4. Lane Changes (`Lane Changes`)

- Data are sorted by License Plate and Timestamp.
- The current lane is compared with the previous one. If they differ (and it is the same plate), it is counted as a change.
- The rate is normalized by $(N_P - 1)$.

9.2.5 5. Weighting (If applicable)

If `monthly_weighting` is enabled, metrics are computed first by grouping by Plate-Month, and then a weighted average is taken using the number of trips in the month to obtain the final value for the plate.

10 Scope and methodology

This report documents and audits the GNN pipeline implemented in the repository, focusing on:

- The tab-by-tab workflow of the Streamlit application (`src/graph_builder_app.py`).
- Graph construction, the GNN model, training, and evaluation (`src/graph.py`, `src/gat_model.py`, `src/gnn_main.py`, `src/train_pretrain.py`).
- Class balancing mechanisms: GraphSMOTE and ImGAGN (`src/graphsmote.py`, `src/imgagn.py`).
- Hyperparameter optimization (Optuna) and metric tracking.

The audit includes best-practice checks, information leakage risks, memory/compute considerations, and consistency with configuration options in `src/config.py`.

11 Overall pipeline map

Conceptual summary:

1. **Base data**: gantries, flows, and accidents (`src/utils.py`).
2. **Feature engineering**: snapshot/flow processing (`src/snapshot_pipeline.py`, `src/features.py`).
3. **Feature selection**: manual/UI-guided selection and persistence.
4. **Graph**: `pm` nodes, `temporal/spatial/st_fwd` edges, and edge attributes (`src/graph.py`).
5. **Training**: HeteroGAT + optional TemporalAggregator (`src/gat_model.py`, `src/temporal_head.py`).
6. **Balancing**: GraphSMOTE (embedding-space + z→x + edge generation) and/or ImGAGN (adversarial) (`src/graphsmote.py`, `src/imgagn.py`).
7. **Evaluation**: metrics (F1, AUC, AUPRC, MCC), thresholds, and calibration (`src/gnn_main.py`).
8. **Artifacts**: models, hparams JSON files, augmented graphs, and run history (`Resultados/`, `gnn_history.jsonl`).

12 Pipeline *tab by tab* (Streamlit Graph Builder)

The UI flow is implemented in `src/graph_builder_app.py` and organized in tabs.

12.1 Events

UI function: `_render_events_tab`.
Goal: load and validate accidents, normalize columns, clean data, and prepare the events dataset.

- Load accidents from CSV/selected files and persist them in `st.session_state`.
- Use helpers in `src/utils.py` for column and timestamp normalization.
- Support generation of *Black Spot* reports (later used in *Graph*).

Implication: this tab defines the accident universe used by later labels and temporal splits.

12.2 Feature Engineering

UI function: `_render_feature_engineering`.
Goal: generate or load features per `pm` node.

- Main modes: *Snapshot Features* and *Flow 5 min* (see `src/snapshot_pipeline.py`).
- Feature generation via `compute_pm_features` (`src/features.py`) and/or `SnapshotFeatureBuilder`.
- Output persisted in DuckDB or CSV, synchronized by time window and road segment.

Implication: affects node dimensionality and feature distribution. Time resolution is controlled with `DT_MINUTES` and parameters in `src/config.py`.

12.3 Feature Explorer

UI function: `render_feature_explorer`.
Goal: visual inspection, descriptive statistics, and outlier diagnostics for input variables. **Implication**: quality gate for feature sanity before graph construction.

12.4 Feature Selection

UI function: `_render_feature_selection_tab`.
Goal: select feature subsets for training and persist selected lists/metadata. **Implication**: changes the GNN input space and `pm.x` dimensionality.

12.5 Graph

UI functions: `_render_create_graph`, `_render_load_graph`, `_render_in_memory_graph`.
Goal: build or load the heterogeneous gantry graph.

- Segment selection (manual or black-spot-driven).
- Construction of `pm` (gantry-minute) nodes with normalized features.
- Generation of temporal/spatial edges and edge attributes (see Section 13).
- Creation of `train/val/test_mask` via temporal cutoffs (`_compute_time_cutoffs`).
- Save to `Resultados/highway_graph_*.pt` and memory (`st.session_state`).

12.6 Visual Graph

UI function: `render_visual_graph_tab`.
Goal: visualize topology, degree patterns, and subgraphs for QA.

12.7 Network

UI function: `_render_network_tab`.
Goal: configure model architecture and parameters before training. **Implication**: sets values such as `hidden_channels`, `num_heads`, `dropout`, `num_layers`, `aggr1/aggr2`, and checkpointing.

12.8 Optuna

UI function: `_render_optuna_tab`.
Goal: run hyperparameter optimization with Optuna (see Section 18). **Implication**: stores `optuna_hyperparams_*.csv` and drives auto-selection during training.

12.9 Balance

UI function: `_render_balance_tab`.
Goal: generate balanced graphs with GraphSMOTE or ImGAGN.

- GraphSMOTE: requires a trained model for embeddings and `z→x` decoders.
- ImGAGN: trains adversarial generator and discriminator to create synthetic nodes.

Implication: creates graphs with `is_synthetic`, updates `train_mask`, and can modify effective class balance.

12.10 Training

UI function: `_render_training_tab`.
Goal: train the GNN with optional balanced graph, early stopping, and automatic evaluation.
Backend: `src/gnn_main.py::run_gat_training` and `src/train_pretrain.py::train_minibatch`.

12.11 Evaluation

UI function: `_render_evaluation_tab`.
Goal: load models, evaluate metrics, apply thresholding, and optionally calibrate probabilities.
Backend: `src/gnn_main.py::test` and export utilities.

12.12 History

UI function: `_render_history_tab`.
Goal: experiment registry and traceability (`Resultados/gnn_history.jsonl`).

12.13 Experiments

UI function: `_render_gnn_experiments_tab`.
Goal: run experiment batches, export results, and import ZIP bundles.

13 Graph data model

Nodes: `pm` (gantry-minute).
Targets: `pm.y` (binary label), `pm.is_accident_pm` (direct accident flag).
Masks: `train_mask/val_mask/test_mask/sequence_mask`.

13.1 Edges and attributes

Main edges (`src/graph.py`):

- `temporal`: $P_{i,t} \to P_{i,t+1}$ (same location, next time).
- `spatial`: $P_{i,t} \to P_{i+1,t}$ (next gantry, same time).
- `st_fwd` (optional): $P_{i,t} \to P_{i+1,t+1}$.
- `spatial_back` (optional): reverse of `spatial`.

Edge attributes: feature differences between destination and source (Δx) plus extra channels for `spatial` edges (gradients, shock, distance). For `temporal` and `st_fwd`, extra channels are set to zero to keep dimensions aligned.

13.2 Temporal labeling

Time cutoffs: defined from chronologically sorted accidents (`_compute_time_cutoffs`).
Masks: assigned by minute-level timestamps.
SequenceIndex: temporal sequences with guard band and future horizon (`src/snapshot_sequences.py`).

14 GNN model architecture

Base model: `src/gat_model.py::HeteroGAT`.

- **Layers**: `num_layers` blocks of `HeteroConv` with `GATConvSaveAlpha`.
- **Aggregation**: `aggr1` on first layer, `aggr2` on subsequent layers.
- **Attentions**: captured when `XAI=1`, used for $L2$ regularization.
- **Edge attributes**: validated per batch; missing/misaligned tensors are zero-padded.
- **Checkpointing**: `use_checkpointing` reduces memory usage via recomputation.

14.1 Temporal head (optional)

Class: `src/temporal_head.py::TemporalAggregator`.
Idea: GRU over recent embedding sequences; keeps a cache for nodes not present in the current batch, avoiding gradient leakage.

14.2 GNN variant registry (encoder + temporal + edge processing)

The pipeline now includes a **variant registry** controlled from `src/config.py` (`GNN_VARIANT`, `GNN_TEMPORAL_CACHE_` and instantiated in `src/gnn_main.py` through `_build_gnn_model`, `_build_temporal_head_for_variant`, and `_prime_temporal_cache_if_needed`. In addition, the UI in `src/graph_builder_app.py` exposes a condensed guide in the *Network*, *Training*, *Optuna*, and *Evaluation* tabs. The design separates:

- **Spatial encoder**: base heterogeneous GAT, GAT with edge encoder, or edge-aware alternative.
- **Temporal head**: snapshot-only (no sequence), GRU, temporal Transformer, or attention pooling.
- **Cache protocol**: batch-wise `incremental` caching or `global_epoch` (one global encoder pass per epoch for consistent sequences).

14.2.1 General comparability idea

All variants share the same `SequenceIndex` (built in `src/snapshot_sequences.py` using `sequence_rows`/`target_ro` the same graph, and the same temporal splits. This enables architectural comparisons without changing the target definition or prediction horizon.

"5 minutes before" labeling The labeling logic uses:

$$\text{positive interval} = (t + \gamma,\ t + \gamma + H]$$

where γ is `GUARD_BAND_MINUTES` and H is `HORIZON_MINUTES`. For a strict "5 minutes before" setup, the recommended configuration is:

- `GUARD_BAND_MINUTES = 5`
- `HORIZON_MINUTES = 5` (or 10–20 for a risk window)

in `src/config.py`.

14.2.2 Implemented variants

Implementation: `src/gat_model.py`, `src/temporal_head.py`, `src/temporal_heads.py`, `src/gnn_main.py`.
Selection: via the `GNN_VARIANT` string (normalized in `src/gnn_main.py::_normalize_gnn_variant`).

1. `gat_snapshot` (baseline): `HeteroGAT` without a temporal head. Classification uses the current snapshot embedding.
2. `gat_gru`: `HeteroGAT` + `TemporalGRUHead` (`src/temporal_heads.py`).
3. `gat_transformer`: `HeteroGAT` + `TemporalTransformerHead` (self-attention with learnable positional embedding).
4. `gat_attnpool`: `HeteroGAT` + `TemporalAttnPoolHead` (lightweight, low-cost ablation).
5. `gat_edge_mlp`: `HeteroGATWithEdgeEncoder` (per-edge-type MLP over `edge_attr`) without a temporal head.
6. `gat_edge_mlp_gru` and `gat_edge_mlp_transformer`: combine edge encoding with temporal heads.
7. `gnn_edge_aware`: `HeteroEdgeAware` (built on `TransformerConv`) without a temporal head.
8. `gnn_edge_aware_gru` and `gnn_edge_aware_transformer`: edge-aware variants with temporal modeling.

14.2.3 Edge attribute encoding (`edge_attr`)

The `HeteroGATWithEdgeEncoder` variant adds an edge MLP (`EdgeAttrEncoder`) before graph attention. Motivation:

- Δx channels may have heterogeneous scales;
- a learned mapping can induce a more stable geometry for attention;
- it enables isolated analysis of "better `edge_attr`" vs. "better temporal head".

14.2.4 Temporal heads and sequence assembly

Temporal heads reuse the `TemporalAggregator` infrastructure:

- `update_cache(...)` for global or partial embedding caching.
- `_build_sequence_batch(...)` to reconstruct $[B, L, D]$ tensors using `sequence_rows`.

This allows changing the temporal operator (GRU/Transformer/attention pooling) without changing the training contract in `src/train_pretrain.py` or the model `forward` signature.

14.2.5 Temporal cache strategies (critical implementation point)

- `incremental`: batch-wise cache updates (historical behavior, lower cost, more sampler-dependent).
- `global_epoch`: epoch-level priming using a global encoder pass (`compute_epoch_embeddings`), then train/eval consume a consistent cache.

Recommendation for paper-grade comparisons: prefer `global_epoch` so temporal sequences are aligned to the same embedding reference within each epoch.

14.2.6 Recommended experiment matrix

1. `gat_snapshot` (spatial baseline)
2. `gat_gru`
3. `gat_transformer`
4. `gat_edge_mlp_gru`
5. `gnn_edge_aware_gru` (or `gnn_edge_aware_transformer`)

Keep seed, temporal split, `SequenceIndex`, and evaluation protocol fixed.

14.2.7 Metrics for rare events

For scientific and operational comparison, prioritize:

- **PR-AUC / AUPRC**: primary metric under severe imbalance.
- **ROC-AUC**: secondary reference.
- **Recall at target FAR/FPR**: useful under operational false-alarm constraints.
- **MCC**: robust under strong class asymmetry.

14.2.8 Variant traceability

Training and HPO persist `gnn_variant` and `variant_tag` in hyperparameter sidecars, checkpoints, and structured logs (`Resultados/gat_model_BEST*.pt`, `_hparams.json`, Optuna CSVs). This prevents mixing checkpoints from incompatible architectures.

15 Training and optimization

Entry point: `src/gnn_main.py::run_gat_training`.

15.1 Training flow

1. **Device**: `get_auto_device()` (MPS > CUDA > CPU).
2. **Data load**: `loaded_obj['data']`.
3. **Automatic ImGAGN** (if `AUTO_IMGAGN_PRETRAIN=1`).
4. **GraphSMOTE**: selected by user/flag with detection of existing synthetic nodes.
5. **HPO**: load `optuna_hyperparams_*.csv` or run a new search.
6. **Model**: the `_build_gnn_model` factory selects the encoder/temporal head according to `gnn_variant` and attaches a temporal head when applicable.
7. **z→x decoders**: train if GraphSMOTE is active (`train_z2x_decoders`).
8. **Edge generator**: `RelEdgeGen` for edge reconstruction and synthetic edge creation.

9. **Loss**: `CrossEntropy` or `FocalLoss` (with or without class weights).
10. **Scheduler**: `OneCycleLR`.
11. **Loop**: mini-batch training, validation, early stopping, and best-model checkpointing.

15.2 Mini-batch training and regularization

Function: `src/train_pretrain.py::train_minibatch`.

- Classification on `pm` nodes (first `batch_size` elements from `NeighborLoader`).
- Total loss:
$$\mathcal{L} = \mathcal{L}_{cls} + \lambda_{edge}\,\mathcal{L}_{edge} + \lambda_{att}\,\mathcal{L}_{att}.$$
- $\mathcal{L}_{edge}$: edge reconstruction with negative sampling (when `edge_gen` is active).
- $\mathcal{L}_{att}$: $L2$ attention regularization when `XAI=1`.
- Gradient accumulation (`accumulation_steps=2`) and `grad_clip`.
- AMP support (CUDA only).

16 GraphSMOTE: synthetic node and edge generation

Module: `src/graphsmote.py`.
Principle: synthesize nodes in embedding space and decode them into feature space.

16.1 Step 1: embeddings

- `get_embeddings_minibatch` (offline UI balancing) or `compute_train_embeddings` (training-time, leakage-safe).
- Node-wise ℓ_2 normalization.

16.2 Step 2: SMOTE in embedding space

SMOTE is applied to the minority set:

$$\mathbf{z}_{syn} = (1-\alpha)\,\mathbf{z}_i + \alpha\,\mathbf{z}_j, \quad \alpha \sim U(0,1).$$

- k-NN over minority embeddings (CPU cache or GPU when feasible).
- Key parameters: `GRAPHSMOTE_K`, `smote_k`, `random_state`.

16.3 Step 3: z→x decoders

Class: `Z2XDecoders`.
Training: `train_z2x_decoders` with SmoothL1/MSE, early stopping, and validation.

- One MLP per node type: $\mathbf{x}\_syn = f_{ntype}(\mathbf{z}\_syn)$.
- Respect `train/val_mask` and avoid out-of-split nodes when masks are available.

16.4 Step 4: node insertion

- Concatenate synthetic `x` and `y` into `pm.x` and `pm.y`.
- Update `train/val/test_mask` (new nodes go to train by default).
- Create/update `is_synthetic` and `is_accident_pm`.

16.5 Step 5: synthetic edge generation

Generator: `RelEdgeGen` (relation-specific parameters).
Real-node candidates:

- `GRAPHSMOTE_CONECT=1`: top-K within `train_mask`.
- `GRAPHSMOTE_CONECT=0`: one-hop neighbors of real accident nodes.

Direction:

- `BIDIRECCTION=1`: real $\leftrightarrow$ synthetic.
- `BIDIRECCTION=0`: real $\rightarrow$ synthetic only.

Edge attributes: computed as destination-source Δx (with `delta_feature_idx` when available) and aligned to existing edge-attribute dimensionality.

16.6 Operation modes

- **Offline**: `augment_graph_offline_once` augments once and trains on the augmented graph.
- **Online**: `refresh_synthetics_online` regenerates nodes every `SMOTE_EVERY_N_EPOCHS`.
- **None**: no augmentation.

17 ImGAGN: adversarial generation

Module: `src/imgagn.py`.
Core idea: generate synthetic nodes as convex combinations of minority *training* nodes and train an adversarial discriminator.

17.1 Step 1: homogenization

`to_homogeneous_if_needed` converts `HeteroData` into a homogeneous graph (target `pm`), preserving `edge_attr` and `delta_feature_idx` when present.

17.2 Step 2: define synthetic node count

$$\lambda_1 = \frac{n_{min} + n_g}{n_{maj}}$$
$$\Rightarrow n_g = \max\big(0,\, \lambda_1 n_{maj} - n_{min}\big)$$

The value is capped by `max_new_nodes` to avoid OOM conditions.

17.3 Step 3: generator

Class: `ImGAGNGenerator`.

- Input: noise $\mathbf{z} \sim \mathcal{N}(0, I)$.
- Output: logits over minority training nodes.
- Weights $T = \text{softmax}(G(z))$ and synthetic features $X_g = TX_{min}$.
- Edges: top-K links to real minority nodes according to T.

17.4 Step 4: discriminator

Class: `ImGAGNDiscriminator` (GCN + two heads).

- **Real/fake head** (BCE).
- **Minority/majority head** (BCE).
- Separation term `_mm_separation` to push minority and majority means apart.

17.5 Step 5: adversarial training

For each `epoch`:

1. Sample noise and build synthetic nodes (without gradients).
2. Build augmented graph and train D for `d_steps` mini-batches.
3. Recompute X_g with gradients and optimize G using:

$$\mathcal{L}_G = \mathcal{L}_{rf} + \mathcal{L}_{mi} + \mathcal{L}_{di},$$

   where $\mathcal{L}_{rf}$ pushes toward *real*, $\mathcal{L}_{mi}$ pushes toward *minority*, and $\mathcal{L}_{di}$ pulls embeddings toward minority centroids.

Scalability: uses `NeighborLoader` when available; otherwise falls back to full-batch.

17.6 Step 6: final graph

The augmented graph is rebuilt with the final generator, returning: `x_aug`, `edge_index_aug`, `edge_attr_aug`, and the generated-node block.

18 Optuna (hyperparameter optimization)

Function: `src/gnn_main.py::objective`.

- Objective: maximize validation F0.5 (precision-oriented).
- Search space: `hidden_channels`, `num_heads`, `dropout`, `num_layers`, `aggr1/aggr2`.
- Optimizers: Adam/AdamW/RAdam (configurable).

- With GraphSMOTE: search `smote_k`, `target_pos_ratio`, `smote_every_n_epochs`, `lambda_edge`.
- Without GraphSMOTE: `FocalLoss` with `focal_alpha`, `focal_gamma`.

Seed sampling options (train/val):

- *Resample seeds each epoch*: changes subset every epoch (robustness-oriented).
- *Fix subset per trial*: keeps one subset during each trial (lower variance).
- *Load full graph in memory (no sampling)*: uses all `train/val` nodes (equivalent to `sample_train_nodes=0` and `sample_val_nodes=0`); higher memory usage.

19 Evaluation, calibration, and artifacts

- **Evaluation**: `test` computes F1, AUC, AUPRC, MCC, and confusion matrix.
- **Threshold**: `pick_tau_fbeta` searches threshold τ on validation.
- **Calibration**: optional Platt scaling (`AUTOCALIBRATE_PROBS`).
- **Persistence**: models `gat_model_BEST*.pt`, `GraphSMOTE_embeddings_model*.pt`, and sidecar `_hparams.json`.
- **Graph hash**: `_calculate_graph_hash` for traceability.

20 Technical audit and best practices

20.1 Observed strengths

- Temporal split for `train/val/test` plus `sequence_mask` reduces temporal leakage.
- Training-time GraphSMOTE uses **train** embeddings (`compute_train_embeddings`).
- `NeighborLoader` bounds memory and enables large-graph training.
- Cleaning/*coalesce* mechanisms prevent `edge_attr` inconsistencies.
- Hparams and *run_id* persistence improve reproducibility.

20.2 Risks and critical points (updated)

- **Leakage in manual balancing (mitigated)**: `run_graphsmote` now restricts embedding/minority pools to `train_mask`. Keep auditing this condition.
- **Dependence on `BIDIRECCTION`**: synthetic edge direction affects message flow and may bias signal propagation. Kept unchanged for now.
- **Gradient accumulation (tunable)**: `accumulation_steps` is now configurable (config/UI) but still requires hardware-specific tuning.
- **Balancing + class weights**: class weights are disabled when GraphSMOTE/ImGAGN are active (correct), but validation metric monitoring remains required.

20.3 Prioritized recommendations (current status)

1. **Implemented**: in manual balancing (`run_graphsmote`), restrict to `train_mask` to avoid potential leakage.
2. **Implemented**: expose `accumulation_steps` in config/UI and tune by available memory.
3. **Pending/decision**: keep `BIDIRECCTION` unchanged for now; document rationale and evaluate AUPRC/F1 impact if changed later.
4. **Implemented**: in ImGAGN, register `best_train_recall` together with seed and config for traceability.

21 Appendix: file and function references

- Main UI: `src/graph_builder_app.py`.
- Graph construction: `src/graph.py`.
- GNN model: `src/gat_model.py`.
- Temporal head: `src/temporal_head.py`.
- Training/evaluation: `src/gnn_main.py`, `src/train_pretrain.py`.
- GraphSMOTE: `src/graphsmote.py`.
- ImGAGN: `src/imgagn.py`.
- Configuration: `src/config.py`.

22 MA-AIRL Implementation

22.1 Objective and scope

A functional version of the Multi-Agent Adversarial Inverse Reinforcement Learning (MA-AIRL) algorithm was implemented following the framework described in `src/MAIRL.pdf`. The objective is to learn, for each driver cluster, a stochastic policy and a reward function that explain the observed demonstrations in aggregated cluster data, and then export *vType* parameters for SUMO.

22.2 Overall architecture

The implementation is split into two layers:

- **MA-AIRL core** in `src/marl_core.py`: expert data, normalization, networks (policy, reward, and potential), training, and parameter export.
- **Interface** in `src/multi_agent_rl_app.py`: data loading, *feature* selection, hyperparameter controls, training execution, and metrics visualization.

22.3 Expert data and state construction

Input data are cluster CSV files with the `cluster_label` column. Each row is interpreted as an expert sample, with a state s composed of selected numeric columns (by default: `avg_speed_kmh`, `avg_headway_s`, `lane_change_rate`). The implementation includes:

- **Feature selection** from the UI to define the state vector.
- **Standardization** with a dedicated `StandardScaler` (column-wise mean and standard deviation).
- **Per-agent split** by grouping rows with `cluster_label`.

22.4 Expert actions (SUMO proxy)

MA-AIRL requires (s, a) pairs. Since the CSV data do not contain real actions, an **action proxy** is built for each sample:

- Metrics are mapped to car-following parameters: `maxSpeed`, `accel`, `decel`, `minGap`, `sigma`, `impatience`.
- Heuristic transforms are applied and clipped to `PARAM_BOUNDS` limits.
- Actions are normalized to $[-1, 1]$ for stable training.

This enables MA-AIRL training even without explicit action trajectories.

22.5 MA-AIRL models

For each agent (cluster), three networks are instantiated:

- **Gaussian policy** $q_\theta(a \mid s)$: MLP producing μ and $\log \sigma$.
- **Reward estimator** $g_\omega(s, a)$: MLP over state-action pairs.
- **Potential** $h_\phi(s)$: MLP over the state.

The structured form is implemented as:

$$f_{\omega,\phi}(s, a, s') = g_\omega(s, a) + \gamma h_\phi(s') - h_\phi(s)$$

22.6 Discriminator function and objective

Following Equation (11) in the paper, the discriminator is defined as:

$$D_\omega(s, a) = \frac{\exp(f_\omega(s, a))}{\exp(f_\omega(s, a)) + q_\theta(a \mid s)}$$

Training uses:

- **Discriminator step**: maximize $\mathbb{E}_{X_E}[\log D] + \mathbb{E}_{X_\pi}[\log(1 - D)]$.
- **Generator (policy) step**: maximize $\mathbb{E}_{X_\pi}[f_\omega(s, a) - \log q_\theta(a \mid s)]$ (Equation 12).

22.7 Temporal proxy

The data do not include explicit temporal transitions (s, s'). To keep the MA-AIRL structure, a **one-step proxy** is used:

$$s' = s$$

This preserves the potential-based form without requiring full sequences. The approximation is explicitly documented in code.

22.8 Training metrics

During training, the following metrics are reported:

- **disc_loss**: discriminator loss.
- **policy_loss**: policy loss.
- **reward_mean**: average of f_ω on expert samples.

Metrics are kept in memory for plotting and post-run analysis.

22.9 Export of parameters to SUMO

At the end, per-agent parameters are obtained by evaluating the policy on the mean state and de-normalizing to SUMO ranges. The process exports:

- `mairl_vtypes.add.xml` with one `vType` per cluster.

This enables injecting learned parameters into real SUMO simulations.

22.10 UI integration

An algorithm selector was added:

- **MA-AIRL (paper)**: adversarial training with expert data.
- **CEM (legacy)**: previous cross-entropy method with SUMO.

Hyperparameter controls were also added (batch size, γ, *learning rates*, network size, and device).

22.11 Modified files

- `src/marl_core.py`: MA-AIRL core (data, networks, training, export).
- `src/multi_agent_rl_app.py`: UI and MA-AIRL training flow.

22.12 Current limitations and next steps

- The implementation uses $s' = s$ due to missing explicit temporal sequences.
- Expert actions are heuristic proxies derived from aggregated metrics.
- Recommended next step: build real trajectories from SUMO or time-based data to estimate s' and real actions.